Automaten und Sprachen | AutoSpr

Zusammenfassung

INHALTSVERZEICHNIS

1. Prädikate **2**
1.1. Normalformen 2
1.2. Negation 2
2. Mengen **2**
2.1. Konstruktion 2
2.2. Operationen 2
2.3. Tupel 2
3. Beweise **2**
4. Sprachen **3**
4.1. Wortlänge 3
4.2. Deterministische Endliche Automaten (DEA) 4
4.3. Wörter einer regulären Sprache 4
4.3.1. Übergangsfunktionen für Wörter 4
4.3.2. Wort akzeptieren 4
4.3.3. Akzeptierte Sprache, reguläre Sprache 4
4.4. Myhill-Nerode Automat 4
4.5. Minimalautomat 5
4.6. Pumping Lemma 5
4.6.1. Beweis 6
4.7. Nichtdeterministische endliche Automaten (NEA) 6
4.7.1. Umgang mit mehrdeutigen Übergängen 7
4.7.2. Thompson-NEA 7
4.7.3. $\mathbf{NEA}_{\varepsilon}$ 8
4.8. Mengenoperationen 8
4.9. Reguläre Operationen 9
4.9.1. Alternative 9
4.9.2. Verkettung 10
4.9.3. *-Operation 10
4.10. Reguläre Ausdrücke 10
4.10.1. VNEA 10
4.10.2. Teststrategie 12

1. PRÄDIKATE

Prädikate sind Aussagen über mathematische Objekte, die wahr oder falsch sein können. «Funktionen» mit booleschen Rückgabewerten: $P, Q(n), R(x, y, z)$. Siehe DMI .

1.1. NORMALFORMEN

Normalformen (allgemein, ***kanonische Formen***) helfen, das Vergleichsproblem zu lösen (ob zwei Aussagen dieselben sind).

$$P_1 \wedge \ldots \wedge P_n = \forall i \in \{1, \ldots, n\}(P_i) = \bigwedge_{i=1}^{n} P_i$$

$$P_1 \vee \ldots \vee P_n = \exists i \in \{1, \ldots, n\}(P_i) = \bigvee_{i=1}^{n} P_i$$

1.2. NEGATION

Nicht für alle = Es gibt einen Fall, für den nicht

$$\neg\forall i \in \{1, \ldots, n\}(P_i) \Leftrightarrow \neg(P_1 \wedge \ldots \wedge P_n) \Leftrightarrow \neg P_1 \vee \ldots \vee \neg P_n \Leftrightarrow \exists i \in \{1, \ldots, n\}(\neg P_i)$$

2. MENGEN

2.1. KONSTRUKTION

Grundmenge G, Prädikat $P(x)$. Konstruierte Menge $A = \{x \in G \mid P(x)\}$

2.2. OPERATIONEN

Begriff	***Bedeutung***
Vereinigung	$A \cup B = \{x \mid x \in A \vee x \in B\}$
Schnittmenge	$A \cap B = \{x \mid x \in A \wedge x \in B\}$
Komplement	$\overline{A} = \{x \mid x \notin A\}$
Differenz	$A \setminus B = \{x \in A \mid x \notin B\}$

2.3. TUPEL

$$A \times B = \{(a, b) \mid a \in A \wedge bnB\}$$

n-Tupel:

$$\bigtimes_{i=1}^{n} A_i = \{(a_1, a_2, \ldots, a_n) | a_i \in A_i\}$$

3. BEWEISE

Eine Folge von logischen Schlüssen, die zeigen, dass die Aussage aus den gegebenen Voraussetzungen folgt.

Beweistechnik	***Anwendungsfall***
Konstruktiver Beweis	Liefert explizite «Lösung» / Algorithmus
Widerspruchsbeweis	Geeignet für Unmöglichkeitsaussagen. Z.B. $\sqrt{2} \notin \mathbb{Q}$
Vollständige Induktion	Für Aussagen, die für alle $n \in \mathbb{N}$ gelten.

4. SPRACHEN

Begriff	*Bedeutung*
Alphabet	Eine nichtleere endliche menge Σ heisst ***Alphabet***. Die Elemente von Σ heissen ***Zeichen***.
Wort	Eine Zeichenkette der Länge n ist ein n-Tupel in $\Sigma^n = \Sigma \times \ldots \times \Sigma$. Ein Element von Σ^n heisst ***Wort*** der Länge n. Wörter haben immer endliche Länge.
Leeres Wort	Die Zeichenkette $\varepsilon \in \Sigma^0 = \{\varepsilon\}$ der Länge 0 heisst das ***leere Wort***.
Menge aller Wörter	Leeres Wort ist **immer** drin $$\Sigma^* = \{\varepsilon\} \cup \Sigma \cup \Sigma^2 \cup \Sigma^3 \cup \ldots = \bigcup_{k=0}^{\infty} \Sigma^k$$
Abgekürzte Schreibweisen von Wörtern	$(A, u, t, o, S, p, r) = AutoSpr$ $a^3b^5 = aaabbbbb$ $b^5a^3 = bbbbbaaa$
Sprache	Teilmenge $L \subset \Sigma^*$

4.1. WORTLÄNGE

Begriff	*Bedeutung*
Länge des Wortes	$w \in \Sigma^n \Rightarrow \lvert w \rvert = n$, n ist ***Länge des Wortes*** w
Anzahl Zeichen	Sei $w \in \Sigma^n, a \in \Sigma$. Dann ist $\lvert w \rvert_a$ die ***Anzahl Zeichen*** a im Wort w

Beispiel 1 : **Wortlänge**

$$\lvert \varepsilon \rvert = 0 \qquad \lvert 01010 \rvert_1 = 2 \qquad \lvert a^3b^5 \rvert = 8$$
$$\lvert 01010 \rvert_0 = 3 \qquad \lvert 01010 \rvert_3 = 0 \qquad \lvert (1291)^7 \rvert = 7\lvert 1291 \rvert = 7 \cdot 4 = 28$$

Beispiel 2 : **Sprache**

$$L = \Sigma^* \subset \Sigma^*$$
$$L = \varnothing \subset \Sigma^*, \text{die leere Sprache}$$
$$\Sigma = \{0, 1\}, \text{Sprache aller Binärstrings: } L = \Sigma^*$$
$$\Sigma = \text{Unicode}, J = \{w \in \Sigma^* \mid w \text{ wird vom Java-Compiler akzeptiert}\}$$
$$M \text{ eine "Maschine"}, L(M) = \{w \in \Sigma^* \mid w \text{ wird von } M \text{ akzeptiert}\}$$

Beobachtungen 1

1.1. $\lvert w^n \rvert = n \cdot \lvert w \rvert$

1.2. Zu jeder Maschine M gibt es eine Sprache $L(M)$

4.2. DETERMINISTISCHE ENDLICHE AUTOMATEN (DEA)

Definition

$$A = (Q, \Sigma, \delta, q_0, F)$$

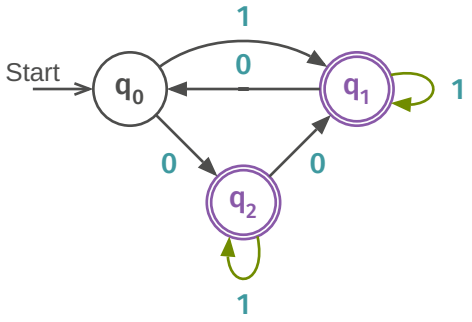
- Zustände: $Q = \{q_0, q_1, \dots, q_n\}$
- Alphabet: Σ
- Übergangsfunktion: $\delta : Q \times \Sigma \to Q$
- Startzustand: $q_0 \in Q$
- Akzeptierzustände: $F \subset Q$

Tabellenform

	0	1	Σ
q_0	q_2	q_1	
q_1	q_0	q_1	δ
q_2	q_1	q_2	

Q, F

Zustandsdiagramm von A



Wichtig:
- Ein Pfeil für jedes Zeichen in jedem Zustand (deterministischer endlicher Automat / DEA)

4.3. WÖRTER EINER REGULÄREN SPRACHE

4.3.1. Übergangsfunktionen für Wörter

$$\delta : Q \times E^* \to Q : (q, w = a_1, \dots, a_n) \mapsto \delta(\dots\delta(\delta(q, a_1), a_2), \dots, a_n)$$

Übergänge ausghend von q für Zeichen $a_1, \dots, a_n$ nacheinander anwenden.

4.3.2. Wort akzeptieren

Der DEA $A = (\Sigma, Q, q_0, \delta, F)$ ***akzeptiert*** das Wort $w \in \Sigma^*$, wenn er A von Startzustand in einen Akzeptierzustand $\delta(q_0, w) \in F$ überführt.

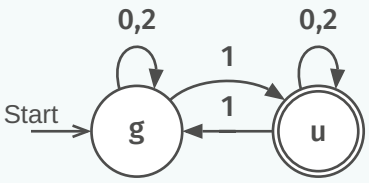
4.3.3. Akzeptierte Sprache, reguläre Sprache

Gegeben ein DEA $A = (\Sigma, Q, q_0, \delta, F)$. Die von A ***akzeptierte Sprache*** ist

$$L(A) = \{w \in \Sigma^* \mid A \text{ akzeptiert } w\} = \{w \in \Sigma^* \mid \delta(q_0, w) \in F\}$$

Die Sprache $L \subset \Sigma^*$ heisst ***regulär***, wenn es einen DEA A gibt mit $L(A) = L$

Beispiel 3 : **DEA, der die Sprache $L = \{w \in \Sigma^* \mid w$ ist eine ungerade Zahl im Dreiersystem$\}$ akzeptiert**



	0	1	2
g	g	u	g
u	u	g	u

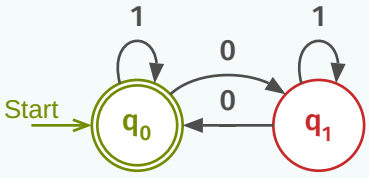
4.4. MYHILL-NERODE AUTOMAT

Für ein Wort $w \in \Sigma^*$ setze $L(w) = \{w' \in \Sigma^* \mid ww' \in L\}$. Insbesondere $L(\varepsilon) = L$

Gegeben: reguläre Sprache L über Σ. Rekonstruiere A mit:
- $Q = \{L(w) \mid w \in \Sigma^*\}$
- $q_0 = L(\sigma) = L$
- $F = \{L(w) \in Q \mid \varepsilon \in L(w)\}$
- $\delta(L(w), a) = L(wa)$

Gleicher Zustand ⇔ gleiches $L(w)$

***Beispiel** 4 :* $\Sigma = \{0,1\}, L = \{w \in \Sigma^* \mid |w|_0 \text{ gerade}\}$



w	$L(w)$	Q
ε	$L(\varepsilon) = L$	q_0
0	$L(0) = \{w \in \Sigma^* \mid \lvert w\rvert_0 \text{ ungerade}\}$	q_1
1	$L(0) = \{w \in \Sigma^* \mid \lvert w\rvert_0 \text{ gerade}\}$	q_0
⋮	⋮	⋮

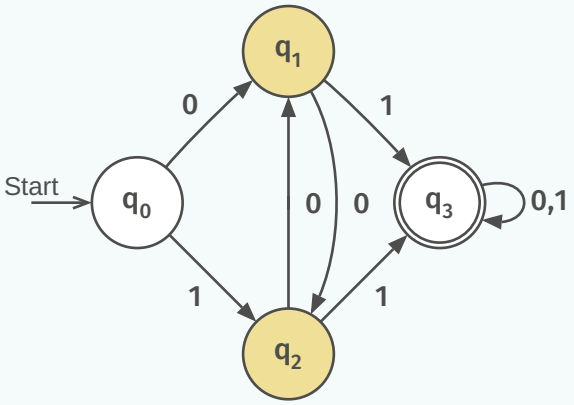
4.5. MINIMALAUTOMAT

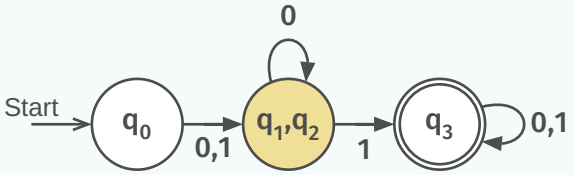
Ziel: Nicht unterscheidbare Zustände zusammenlegen

Vorgehen:

1) Akzeptierzustand ≠ Nichtakzeptierzustand
2) Iteration: alle unterscheidbaren Paare suchen
3) Verbleibende Paare sind ununterscheidbar → zusammenlegen

***Beispiel** 5*





	q_0	q_1	q_2	q_3
q_0	≡	×	×	×
q_1	×	≡	≡	×
q_2	×	≡	≡	×
q_3	×	×	×	≡

Algorithmus

1) $q \in F, q' \notin F$
2) Unterscheidbar nach Übergang
3) Iteration bis keine Änderung mehr

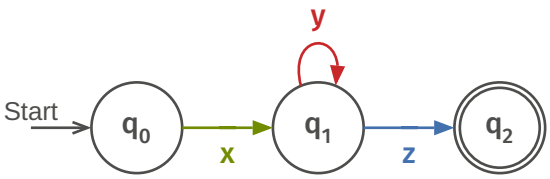
⇒ q_1 und q_2 sind nicht unterscheidbar

4.6. PUMPING LEMMA

Ist L eine reguläre Sprache, dann gibt es $N \in \mathbb{N}$, die pumping length so, dass jedes Wort $w \in L$ mit $|w| \geq N$ in drei Teile $w = xyz$ zerlegt werden kann mit:

1) $|xy| \leq N$
2) $|y| > 0$
3) $xy^kz \in L \forall k \in \mathbb{N}$

Oder: genügend lange Wörter ($|w| \geq N$) einer regulären Sprache ($w \in L$) können alle in einem Anfangsstück der Länge N ($|xy| \leq N$) aufgepumpt werden ($xy^kz \in L$).



Was zeichnet reguläre Sprachen aus?

- Reguläre Ausdrücke
- Lange Wörter: * kommt im regulären Ausdruck vor
- Blöcke mit * können beliebig oft wiederholt werden
- ⇒ Wörter können «aufgepumpt» werden

4.6.1. Beweis

Behauptung: Die Sprache $L \subset \Sigma^*$ ist nicht regulär.

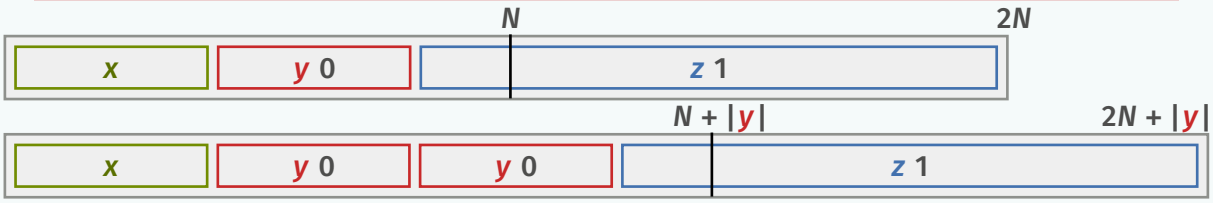
Beweis (Widerspruch):

1) Annahme: L ist regulär
2) Gemäss Pumping Lemma gibt es die Pumping Length N
   - Es darf keine Annahme über die konkrete Grösse von N gemacht werden!
3) Wähle ein Wort $w \in L$ mit $|w| \geq N$
   - Die Definition **muss** N verwenden!
4) Aufteilung des Wortes gemäss Pumping Lemma
   - $w = xyz, |xy| \leq N, |y| > 0$
5) Auswirkung des Pumpens
   - $xy^kz \notin L$ für mindestens ein $k \in \mathbb{N}$ (mit Begründung!)
6) Widerspruch und Schlussfolgerung, dass die Annahme nicht zutreffen kann

Beispiel 6 : **$L = \{0^n1^n \mid n \geq 0\}$ ist nicht regulär**

1) Annahme: L ist regulär
2) $\exists N \in \mathbb{N}$, Pumping Length
3) $w = 0^N1^N$
4) Unterteilung $w = xyz$

TODO:
reduce confusion



5) Pumpen: nur die Anzahl der **1** wird erhöhr, Anzahl **1** bleibt
6) $xy^kz \notin L$ für $k \neq 1$, im Widerspruch zum Pumping Lemma

4.7. NICHTDETERMINISTISCHE ENDLICHE AUTOMATEN (NEA)

Definition

$$A = (Q, \Sigma, \delta, q_0, F)$$

- Endliche Menge von Zuständen: Q
- Alphabet: Σ
- Übergangsfunktion: $\delta : Q \times \Sigma \to P(Q)$
- Startzustand: $q_0 \in Q$
- Akzeptierzustände: $F \subset Q$

Akzeptieren

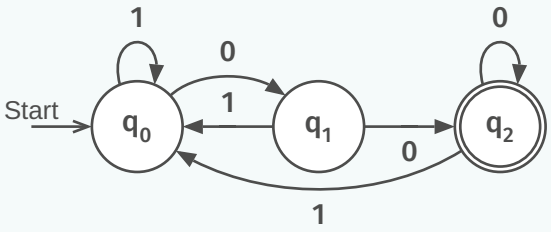
Ein NEA A ***akzeptiert*** das Wort $w \in \Sigma^*$, wenn es eine **Wahl** von Übergängen gibt, derart, dass das Wort w den Automaten in einen Akzeptierzustand überführt.

Faustregeln

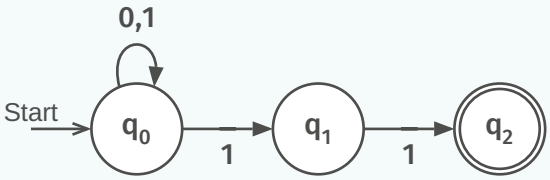
- Nur genau diejenigen Pfeile einzeichnen, die man zum Akzeptieren braucht.
- Erlaube Alternativen

Beispiel 7 : $L = \{w \in \Sigma^* \mid w \text{ endet mit zwei } 0\}$

DEA-Lösung



NEA-Lösung

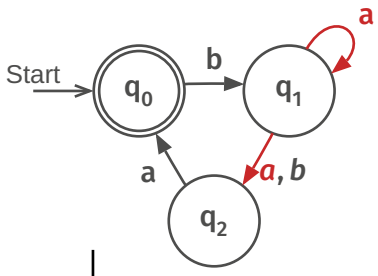


4.7.1. Umgang mit mehrdeutigen Übergängen

a-Übergang von q_1 aus nicht eindeutig

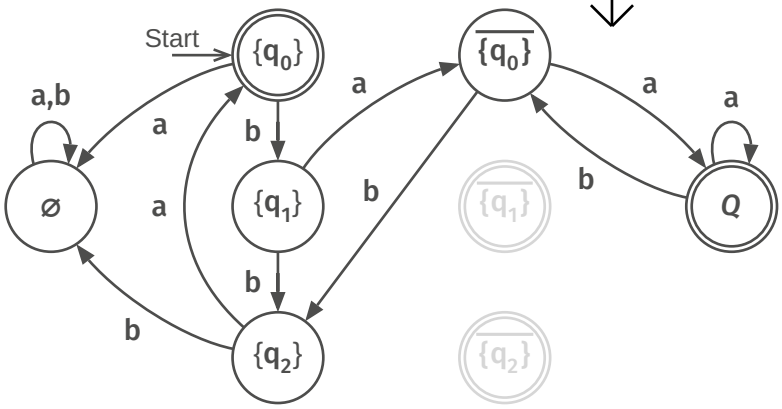
Welchen Übergang soll man nehmen?

- Beide Möglichkeiten probieren und akzeptieren, falls eine der Möglichkeiten auf einen Akzeptierzustand führt.
- Zufallsgenerator (probabilistischer endlicher automat, PEA)



4.7.2. Thompson-NEA

- Zustand = **Menge** der möglichen Zustände
- Akzeptieren = Ob NEA im Zustand sein **könnte**



4.7.2.1. Transformation NEA → DEA

Gegeben $\delta : Q \times \Sigma \to P(Q)$ eines NEA. Übergangsfunktion für Mengen $M \subset Q$

$$\delta' : P(Q) \times \Sigma \to P(Q) : (M, a) \mapsto \delta'(M, a) = \bigcup_{q \in M} \delta(q, a)$$

Satz

Ein NEA A kann in einen DEA A' umgewandelt werden mit

- $Q' = P(Q)$
- $\Sigma' = \Sigma$
- δ' wie oben definiert
- $q'_0 = \{q_0\}$
- $F' = \{M \in P(Q) \mid F \cap M \neq \varnothing\}$

Implementation

Thompson-NEA:

- Zustand $M \subset Q$ realisiert durch Markierung der Zustände in M
- δ' realisiert durch Verschieben von Markierungen
- Akzeptieren: mindestens ein Akzeptierzustand markiert

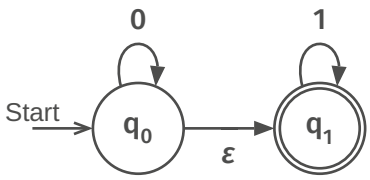
4.7.3. NEA$_\varepsilon$

Begriff	*Bedeutung*
ε-Übergang	Kann ohne Verarbeitung eines Zeichens genommen werden. $$\delta : Q \times (\Sigma \cup \{\varepsilon\}) \to P(Q)$$
NEA$_\varepsilon$	NEA mit ε-Übergängen

Einen NEA$_\varepsilon$ kann man immer in einen NEA umwandeln. Beweis:

1) $E(q)$ = Menge der von q aus mit ε-Übergängen erreichbaren Zustände
2) $E(M) = \bigcup_{q \in M} E(q)$
3) δ ersetzen durch $\delta : Q \times (\Sigma \cap \{\varepsilon\}) \to P(Q) : (q, a) \mapsto E(\delta(q, a))$

$L = \{0^k 1^l \mid k, l \geq 0\}$

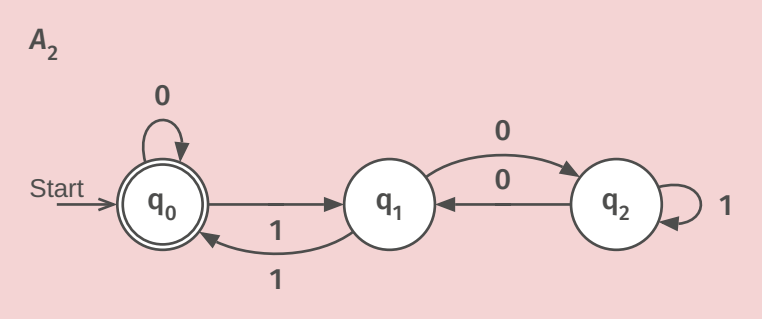


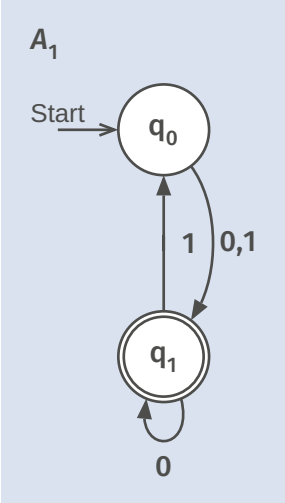
4.8. MENGENOPERATIONEN

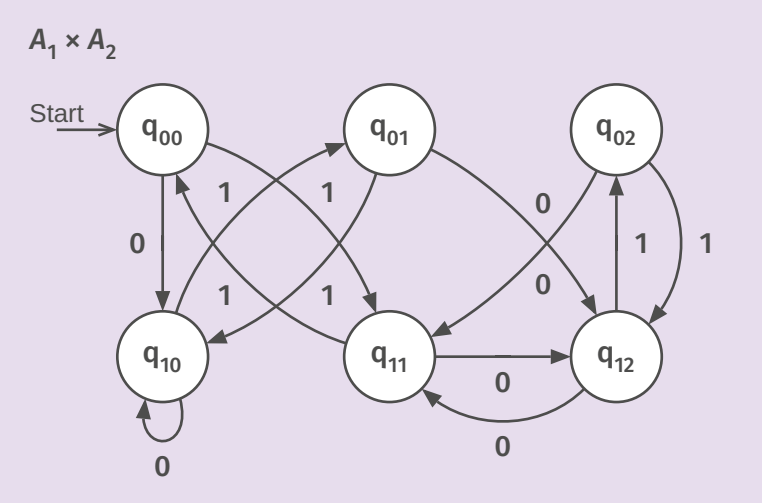
Lassen reguläre Sprachen regulär sein.

Produktautomat: $L_i = L(A_i)$

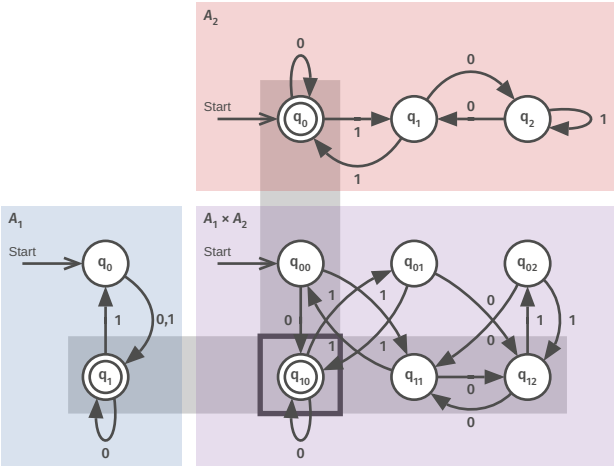
$L = \{w \in \Sigma^* \mid |w|_1 \text{ ungerade}\} \cap \{w \in \Sigma^* \mid w \text{ ist eine durch drei teilbare Binärzahl}\}$



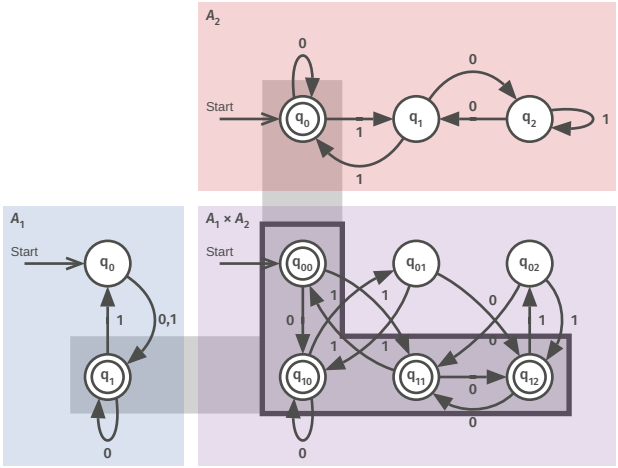




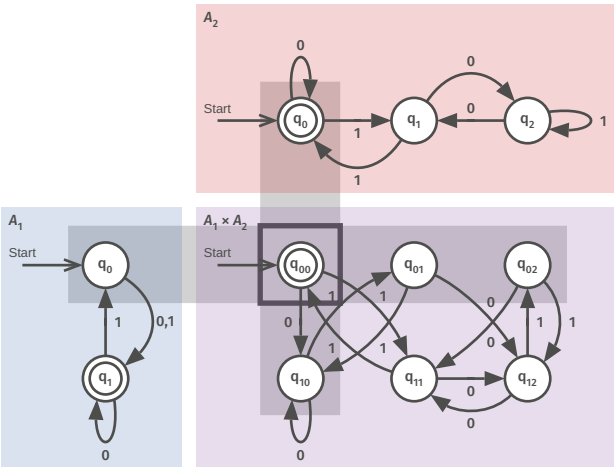
Schnittmenge $L(A_1) \cap L(A_2)$



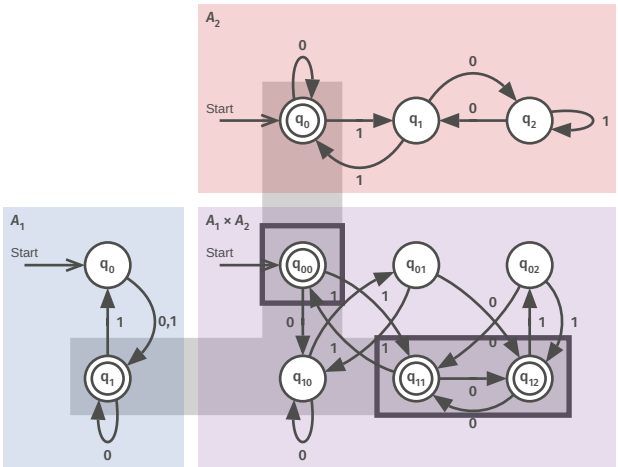
Vereinigungsmenge $L(A_1) \cup L(A_2)$



Differenzmenge $L(A_1) \setminus L(A_2)$



Symmetrische Differenz $L(A_1) \triangle L(A_2)$



Operationen verändern nur Endzustände:

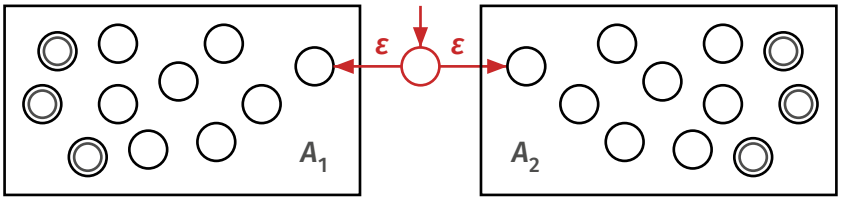
Begriff	***Bedeutung***
Schnittmenge $L_1 \cap L_2$	$F = F_1 \times F_2$
Vereinigung $L_1 \cup L_2$	$F = F_1 \times Q_2 \cup Q_1 \times F_2$
Differenz $L_1 \setminus L_2$	$F = F_1 \times (Q_2 \setminus F_2)$

4.9. REGULÄRE OPERATIONEN

WICHTIG: ε-Übergänge immer hinzufügen

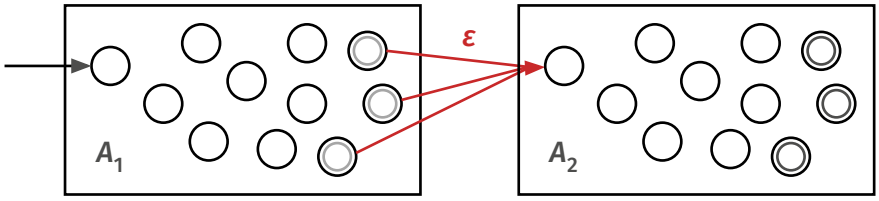
4.9.1. Alternative

$$\begin{aligned} L &= L_1 \cup L_2 \\ &= L(A_1) \cup L(A_2) \\ &= L(A_1) \mid L(A_2) \end{aligned}$$



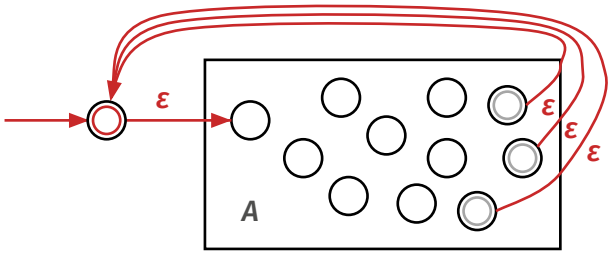
4.9.2. Verkettung

$$L = L_1 L_2$$
$$= L(A_1)L(A_2)$$
$$= \{w_1 w_2 \mid w_i \in L_i\}$$



4.9.3. *-Operation

$$L^* = \{\varepsilon\} \cup L \cup L^2 \cup \ldots$$
$$= \bigcup_{k=0}^{\infty} L^k$$



⇒ die Klasse der regulären Sprachen ist abgeschlossen unter regulären Operationen

4.10. REGULÄRE AUSDRÜCKE

Formeln, die Zeichenketten beschreiben.

1) Buchstaben stehen für sich selbst, mit Ausnahme der Metazeichen **( ) [ ] * ? | . ** (escape character)
2) Verkettung: Zeichen und Formeln hintereinanderschreiben
3) **.** = ein beliebiges Zeichen, **Σ**
4) **|**: Alternative, ***a|A*** = a oder A
5) ***:** Wiederholung, beliebige viele
6) **( )**: Gruppierung
7) **[ ]** Zeichenklassen: **[*abc*] = *a|b|c***, **[^*abc*]** = nicht [abc]

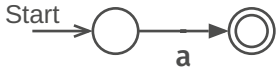
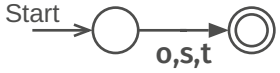
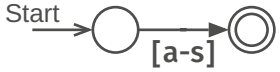
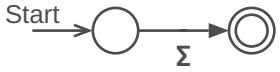
Erweiterungen / Dialekte

1) **{*n*, *m*}**: zwischen ***n*** un ***m*** Wiederholungen
2) **+**: mindestens eines, **{1, }**
3) **?**: optional, **{0, 1}**
4) Symbole für Zeichenklassen: **\ *d*** Ziffern, **\ *s*** whitespace, **[: *space* :]**, **[: *lower* :]**

4.10.1. VNEA

Begriff	*Bedeutung*
Regulärer Ausdruck	Zeichenkette ***r*** zur Beschreibung einer regulären Sprache $L = L(r)$
Reguläre Operationen	$L(r_1) \cup L(r_2) = L(r_1 \mid r_2)$ $L(r_1)L(r_2) = L(r_1 r_2)$ $L(r_1)^* = L(r_1^*)$
Verallgemeinerter NEA	$\mathbf{NEA}_\varepsilon$, dessen Übergänge mit regulären Ausdrücken beschriftet sind

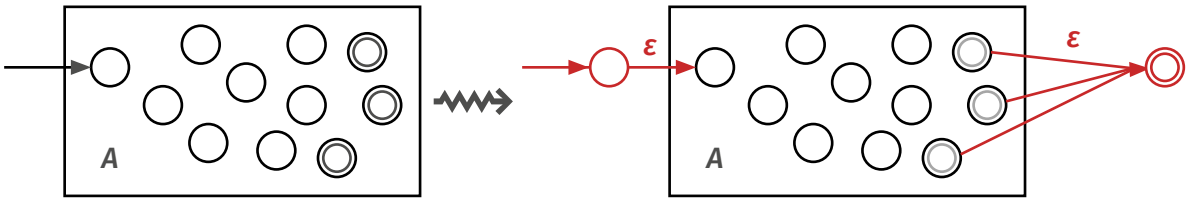
Begriff	***Bedeutung***
Primitive reguläre Ausdrücke	Reguläre Ausdrücke für Wörter mit Länge **≤ 1**

$L = L(r)$	r	NEA
$\varnothing$	$\varnothing$	
$\{\varepsilon\}$	$\{\varepsilon\}$	
$\{a\}$	a	
$\{o,s,t\}$	$[ost]$	
$\{a,b,\ldots,s\}$	$[a-s]$	
Σ	.	

⇒ zu jedem regulären Ausdruck gibt es einen DEA

4.10.1.1. VNEA *A* umwandeln in Regex *r*

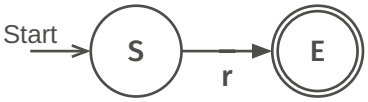
Keine Übergänge nach q_0 und nur ein Akzeptierzustand



Reduktion

Regulärer Ausdruck

Nach Entfernen aller Zwischenzustände $q_{rip} \in Q$ bleibt ein regulärer Ausdruch r von A



⇒ jede reguläre Sprache lässt sich mit einem regulären Ausdruck beschreiben $\{L(A) \mid A \text{ ein DEA}\} = \{L(r) \mid r \text{ ein regulärer Ausdruck}\}$

Interessantes projekt (DEA Lexer DSL): Ragel

4.10.2. Teststrategie

Messung	*Folgerung*
Für $n = 1, 2, 3, \ldots$ – Regulären Ausdruck erzeugen $r = \underbrace{a?\,a?\,a?\,a?\ldots a?}_{n \cdot a?}\underbrace{aaaa\ldots a}_{n \cdot a}$ – Laufzeit für Akzeptieren von a^n durch r messen	– Laufzeit $O(2^n)$: NEA-Implementation – Laufzeit $O(n)$: DEA-Implementation